(12.1) Typical elements of a machine instr:

opcode, source & dest operation references

next instr. ref..

(12.2) source & dest operands ⟶ held in

Register & Memory.

(12.3) If an instr. has 4 addresses, they are:

2 operands

1 result

Address of next instr.

(12.4) 5 instr. set design issues:

1. Operation repertoire: How many & which operations to provide & how complex the " should be.

2 Data type: on which operations are performed.

3 instr. format: Instr. length (in bits),
number of addresses
size of various field etc.

4. Registers: Number of CPU registers that
can be referenced by instructions
+
their use

5 Addressing: Mode/Modes which specified
an operand address.

12.5 typical operands (types:) in Machine instr set:

Address , number,
logical data , characters

12.6 Rltn betn IRA character code & the
packed representation: → 7bit
msB = '011'
0 = 0110000
o = 0111001

For the IRA bit pattern 011XXXX, the digits

0 through 9 are represented by their binary equivalents. 0000 thru 1001, in the right-most 4 bits. Same code as packed decimal

12.7

Arithmetic shift	Logical shift
Designed for signed integer (retain not shift) to preserve sign bit for right shift	Shifts bit left/right, filling vacated positions with zeroes.
Left shift similar to logical shift but may trigger overflow. 0111 (+7) → 1S, 1110 (-2), overflow +ve to -ve sign flip	Treat data as unsigned
RS → shifts all bit to right by 1 pos, the MSB is replicated to preserve the number's sign, LSB discarded. ① 1010 (-6) → RS ① 1101 (-3)	right shift shifts all bit to right by 1 pos. MSB filled with 0, LSB discarded.

12.8 Transfer of control instructions

avoids redundant code, enables complx logic

1. Repetition of instr. seqn (Loops): processing lists, avoiding redundant code ↙ enables executing instr. multiple times

2. Decision Making: Allocos progs to branch based on condn, enabling flexible execution

3 Modular design: organizing code into reusable subroutines (functions)

Transfer of control instructions are essential in programming for three primary reasons:

---

1. Loops (Repetition)
- **Purpose**: To execute a block of code multiple times efficiently.
- **Example**: Processing elements in an array, calculating sums, or iterating through data.

- **Mechanism**: Instructions like `JUMP`, `LOOP`, or conditional branches (e.g., `BNE` in assembly) allow programs to repeat tasks without rewriting code.
  ```assembly
  LOOP:
      ADD R1, R2   ; Perform an operation
      DEC R3       ; Decrement counter
      BNE LOOP     ; Repeat if counter ≠ 0
  ```
- **Why It Matters**: Eliminates redundancy, saves memory, and simplifies maintenance.

---

2. Decision-Making (Conditional Execution)
- **Purpose**: To execute different code paths based on runtime conditions.
- **Example**: Validating user input, handling errors, or choosing algorithms.
- **Mechanism**: Instructions like `BRANCH`, `IF-ELSE`, or `CMP` (compare) enable dynamic branching.
  ```assembly
  CMP R1, R2      ; Compare values
  BGT GREATER     ; Branch if R1 > R2
  ; Code for R1 ≤ R2
  GREATER:
  ; Code for R1 > R2
  ```
- **Why It Matters**: Enables adaptive logic, error handling, and user interactivity.

---

3. Modular Design (Subroutines/Functions)
- **Purpose**: To break complex tasks into reusable, manageable units.
- **Example**: A math function reused across a program (e.g., `sqrt()` or `sort()`).
- **Mechanism**: `CALL` and `RETURN` instructions jump to subroutines and resume execution afterward.
  ```assembly
  MAIN:
      CALL CALCULATE   ; Jump to subroutine
      ; Resume here after RETURN
  CALCULATE:
      ; Subroutine logic
      RET              ; Return to MAIN
  ```
- **Why It Matters**: Promotes code reusability, readability, and simplifies debugging.

Overall Impact
These instructions are the backbone of **structured programming**, enabling:
- **Efficiency**: Avoid redundant code and optimize resource usage.
- **Flexibility**: Respond dynamically to data and user inputs.

- **Scalability**: Manage large, complex programs by modularizing logic.

Without transfer of control, programs would be rigid,

unwieldy, and impractical for real-world tasks. 💻

rather than executing next instr in order, directs cpu to jump to different part of the prog. → load, func, condⁿ, → repeat task, make decision, modularize code.

12.9 2 common way of generating condⁿ to be tested in a condⁿ branch instr.

1) Most machines provide a 1 bit or multiple-bit condⁿ code that is set as the result of some op.

2) For 3 address instr. format → to perform a comparison & specify a branch in the same instr.

so → chatgpt: not solve

① condⁿ codes (flags): A dedicated status register. Many machines update 1 or more condⁿ codes (flags) as a res of arithmetic/logical operatⁿ. (ADD, SUB)

they indicate result like zero, neg, overflow/

carry → used by branch instr to specify next execution path.

After `SUB R1, R2`, zero flag is set if R1 = R2

A BEQ (Branch if Equal) instr checks the zero flag & branches if it is set.

(11) Embedded comparison in Branch instr:

Architecture with 3 addrs instr format:

→ branch instr. can include a comparison operation directly. (allows the instr to compare 2 operands & branch immediately if the specified cond^n is met.

BEQ R1, R2, Label compares R1 & R2 during execution & branches to label if they're equal.

(Due to technical issues, the search service is temporarily unavailable.)

Here are two common ways to generate conditions for conditional branch instructions, explained briefly:

---

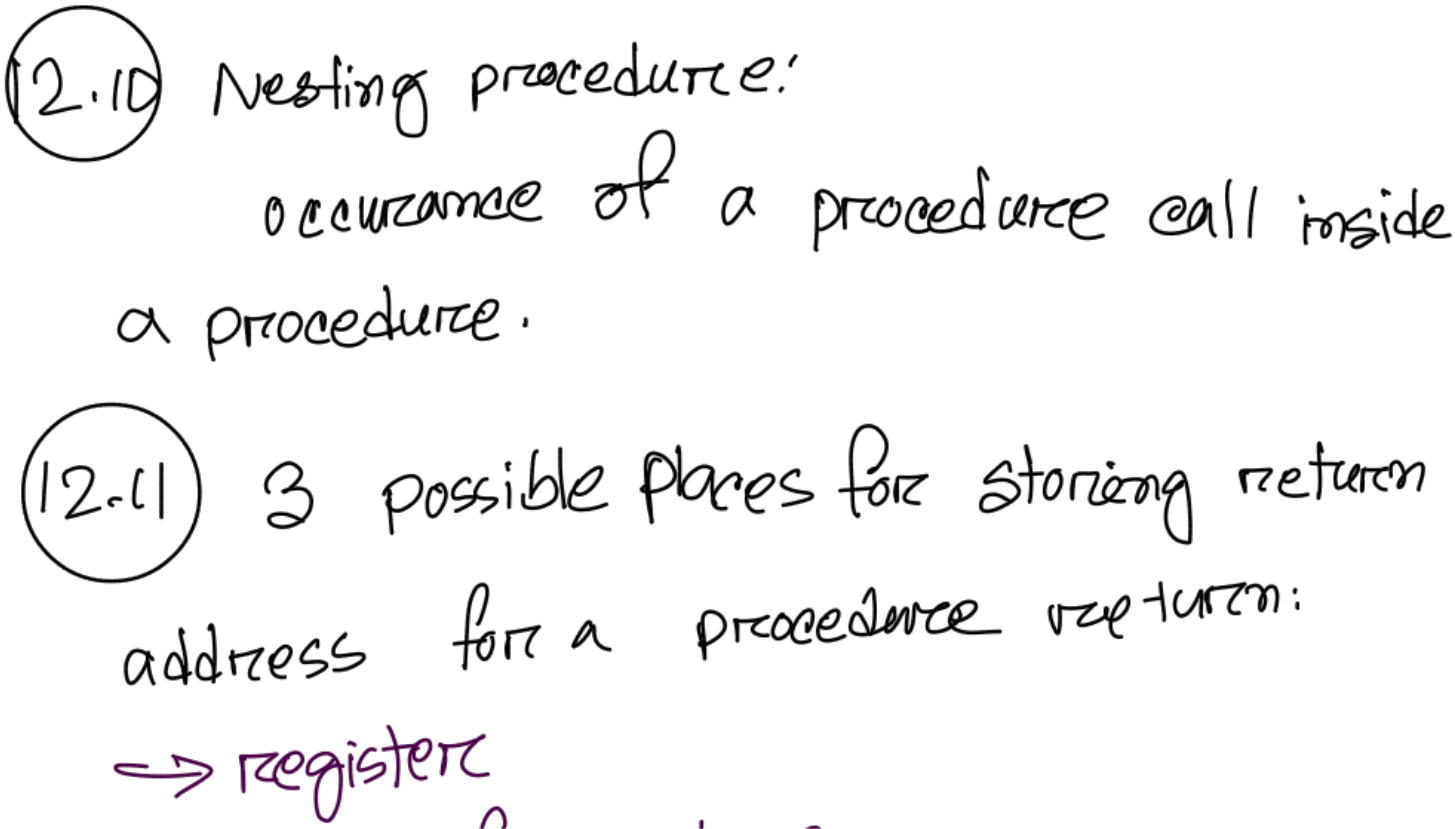
1. **Condition Codes (Flags)**
- **Mechanism**:
  - A dedicated **status register** (e.g., Zero, Carry, Overflow, Negative flags) is updated implicitly by arithmetic/logical operations (e.g., `ADD`, `SUB`).
  - The branch instruction checks these flags to determine whether to jump.
- **Example**:
  - After `SUB R1, R2`, the Zero flag is set if `R1 = R2`.
  - A `BEQ` (Branch if Equal) instruction checks the Zero flag and branches if it is set.
- **Pros**:
  - Flexible: Flags persist and can be reused by subsequent branches.
  - Common in architectures like x86 and ARM.
- **Cons**:
  - Side effects: Every ALU operation alters flags, complicating pipelining and out-of-order execution.

---

2. **Explicit Comparison in Branch Instruction**
- **Mechanism**:
  - The branch instruction **directly compares operands** and decides the branch in a single step (no separate flags).
  - Common in **three-address instruction formats** (two

operands compared, one branch target).
- **Example**:
  - `BEQ R1, R2, Label` compares `R1` and `R2` *during execution* and branches to `Label` if they are equal.
- **Pros**:
  - Simpler control logic: No dependency on a status register.
  - Avoids pipeline stalls caused by flag updates.
  - Used in RISC architectures like MIPS (e.g., `BEQ`, `BNE`).
- **Cons**:
  - Larger instructions: Requires encoding operands and target in one instruction.

Key Difference:
- **Condition Codes**: Relies on persistent flags set by prior operations.
- **Explicit Comparison**: Embeds the comparison within the branch instruction itself.



→ start of procedure

→ top of stack

12.12 Reentrant procedure:
possible to have several calls to open it at the same time.

12.13 reverse polish notation:
the operator follows its 2 operands.

12.14 Big endian vs little endian:

↓

a multi byte numerical val stored with MS byte in the lowest numerical address

↳

MS byte in highest